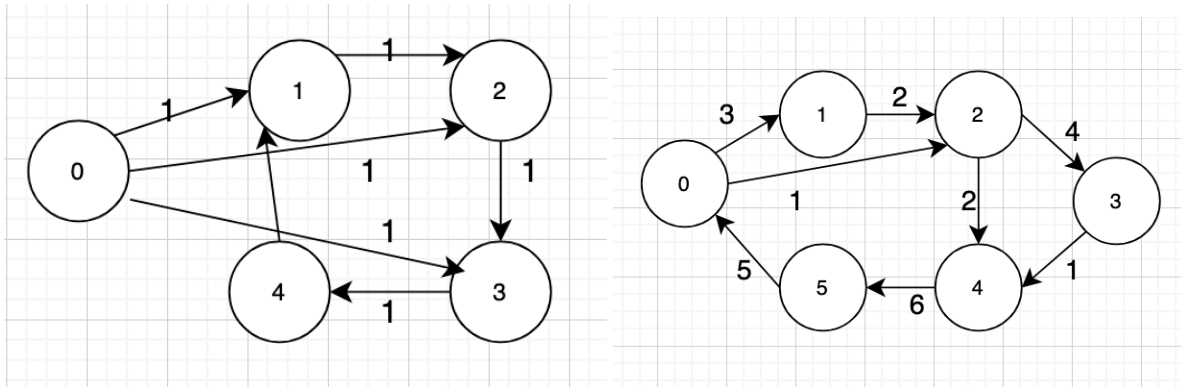
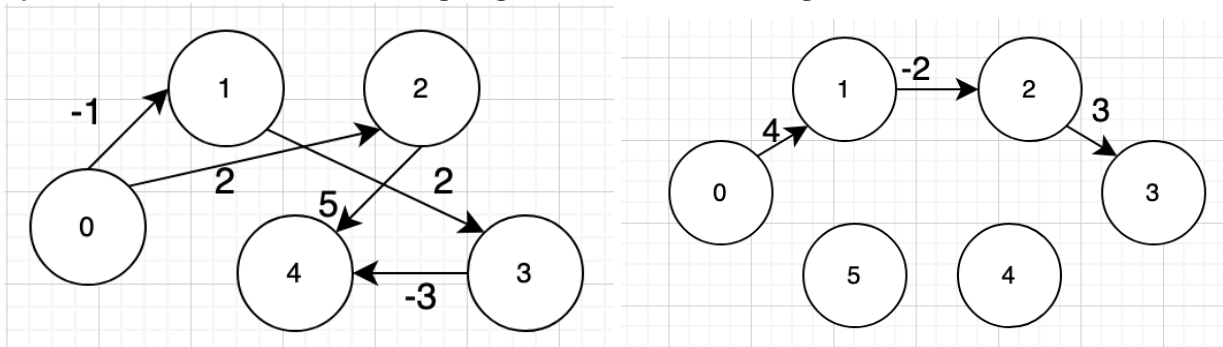
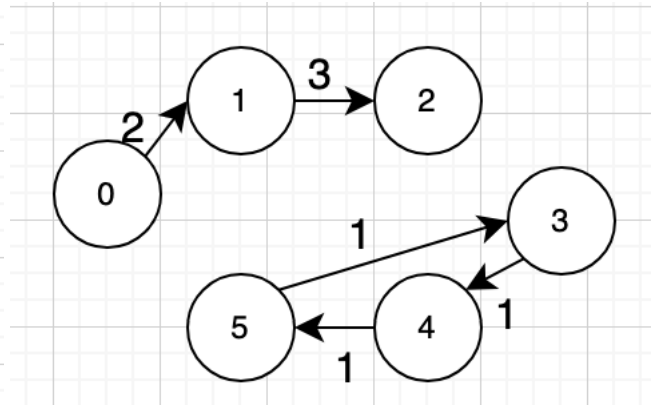
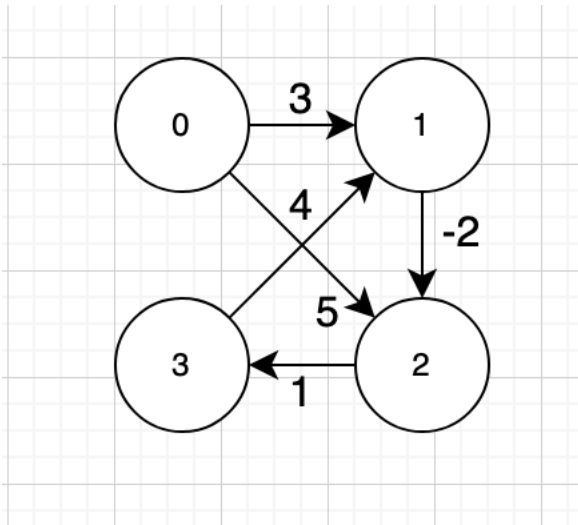




For my two prim test cases I chose a test where all edges are equal lengths and a graph that contains cycles with different edge weights. In the first test, all edges have the same weight, so there are multiple valid MSTs. This tests whether the algorithm can handle situations with equal edge weights and still produce a valid MST. In the second test, the graph contains cycles with edges of different weights. The algorithm must avoid forming cycles in the MST while choosing edges with minimum weights.



For my Bellman-Ford tests I created a test uses a graph with negative edge weights but no negative cycles. It checks whether the algorithm correctly computes the shortest paths in such scenarios as well as a test uses a graph where some nodes are disconnected from the source node. It ensures that the algorithm correctly assigns infinite distance (or maximum integer value) to unreachable nodes. The graph includes negative edge weights but avoids negative cycles, testing the algorithm's ability to handle negative weights correctly. Nodes 4 and 5 are not connected to the rest of the graph, testing whether the algorithm correctly assigns infinite distance to unreachable nodes.



For my Floyd-Warshall I created a case that uses a graph with negative edge weights but no negative cycles as well as a test that contains a graph with a negative cycle. The first test checks whether the algorithm correctly handles negative weights while computing shortest paths. The second test checks whether the algorithm can detect negative cycles and handle them appropriately.

While implementing these solutions the biggest problem I ran into was with the Bellman-Ford algorithm. Here I found it slightly difficult to figure out the best way to retrieve all the edges. To solve this, I simply iterated over every edge and check if there was an edge between them.